

Dennemeyer System Design

100 téma példával, technikai / architect interjúra

Cél: telefonról olvasható, de nem túl száraz jegyzet. Minden témánál van rövid magyarázat és ahol értelmes, konkrét példa. A fókusz: API/Data Exchange Platform, microservices, security, Docker/Kubernetes, SQL, reliability és technical leadership.

Használat: először a címet és a példát nézd. Ha kérdeznek, a rövid magyarázatból bontsd ki. Nem lexikon, hanem interjú előtti mentális térkép.

1-10. témák

1. API design és API contract

Az API nem csak endpointok listája, hanem szerződés a kliensek és a backend között. Stabil request/response forma, validáció, hibakódok, pagination, versioning, auth és dokumentáció kell hozzá. Külső ügyfeleknél ez különösen fontos, mert ők az API contractra építenek.

Példa: Egy POST /renewals endpointnál pontosan dokumentáld a request mezőket, a 201 response-t, a 400/401/403/409 hibákat és azt, melyik mező optional.

Interjú: I treat APIs as contracts: validation, consistent errors, compatibility and clear documentation are important.

2. Data Exchange Platform

Olyan platform, amely különböző rendszerek között adatot fogad, validál, átalakít és továbbít. Tipikus gondok: eltérő formátumok, hiányzó mezők, duplikáció, jogosultság, audit log, retry és hibakezelés. Dennemeyernél ez lehet ügyfélrendszer és belső IP szolgáltatás közötti integráció.

Példa: Egy ügyfél CSV-ben vagy API-n küld trademark renewal adatot. A platform validálja, belső formátumra alakítja, elmenti, majd státuszt ad vissza.

3. Idempotency

Ugyanaz a kérés többször lefutva sem okoz többszörös mellékhatást. Példa: timeout miatt a kliens újraküldi a renewal requestet, de nem jöhet létre két renewal. Megoldás lehet idempotency key, unique constraint vagy processed event tábla.

Példa: A kliens timeout után újraküldi ugyanazt a payment vagy renewal requestet ugyanazzal az idempotency key-jel. A backend nem hoz létre duplikált rekordot.

Interjú: For external APIs and queues I usually assume retries can happen, so writes should be idempotent where possible.

4. Retry, backoff, DLQ

Retry akkor kell, ha átmeneti hiba van. Backoff azt jelenti, hogy nem azonnal és agresszíven próbálkozik újra, hanem növekvő várakozással. DLQ, vagy dead-letter queue, az a hely, ahová a végleg elbukott üzenetek kerülnek későbbi vizsgálatra.

Példa: Ha a külső payment provider 503-at ad, retry mehet 1s, 2s, 4s várakozással. Ha sokszor bukik, az üzenet DLQ-ba kerül.

Interjú: Transient failures can be retried with backoff, but invalid business data should fail fast and be visible.

5. Microservices

A microservice külön deployolható, külön felelősségű service. Akkor jó, ha tiszta ownership, API contract, adat-határ, monitoring és deployment van mögötte. Nem az a cél, hogy minden apró legyen, mert túl sok service növeli a latencyt és a debugging nehézségét.

Példa: Payment, document generation és notification lehet külön service, ha külön skálázódnak és külön csapat/ownership van mögöttük.

6. DDD, Domain-Driven Design

A DDD lényege, hogy a kód a business fogalmak köré szerveződjön. Fontos: bounded context, entity, value object, aggregate, domain service. IP domainben ilyen fogalmak lehetnek: trademark, patent, renewal, portfolio, payment.

Példa: IP domainben a Renewal nem csak DB sor, hanem business fogalom szabályokkal: mikor indítható, ki jogosult rá, milyen státuszai vannak.

Interjú: I first try to understand the domain language, then design modules around clear business concepts.

7. CQRS

CQRS az írási és olvasási modell szétválasztása. Command módosít, query olvas. Akkor hasznos, ha az írás komplex business szabályokat követ, az olvasás pedig gyors dashboard/read model optimalizálást igényel. Nem default megoldás, csak ha a komplexitás indokolja.

Példa: Íráskor komplex renewal workflow fut, olvasáskor viszont egy gyors customer dashboard kell. Ilyenkor lehet külön command model és read model.

Interjú: I would use CQRS only where read and write needs are really different enough to justify it.

8. SOLID

A SOLID objektumorientált design elvek gyűjteménye. Nem definíciókat kell bemagolni, hanem a célt érteni: kisebb felelősségű, lazán kapcsolt, könnyen tesztelhető modulok. Architect szinten ugyanez service-ekre is igaz: tiszta felelősség, kevés rejtett dependency.

Példa: Single Responsibility: a RenewalService ne generáljon PDF-et, ne küldjön emailt és ne kezeljen payment logikát egyszerre. Ezek legyenek külön felelősségek.

9. Docker és Kubernetes

Docker image-be csomagolja az appot. Kubernetes futtatja, skálázza, restartolja és rolloutolja a containereket. Amit tudni kell: deployment, service, ingress, configmap, secret, readiness/liveness probe, rolling update, rollback.

Példa: Új API verzió rolloutja közben Kubernetes először új podokat indít, readiness után küld rájuk forgalmat, majd fokozatosan leállítja a régiakat.

Interjú: I care about deployment safety: readiness checks, graceful shutdown, rolling updates and rollback.

10. Observability

Nem csak logolás. Három fő része: logs, metrics, traces. Cél, hogy production hibánál gyorsan lásd melyik request, melyik user/customer, melyik service és milyen latency vagy hiba okozta a gondot. Correlation ID és structured logging nagyon hasznos.

Példa: Egy hibás customer requestnél correlationId alapján megtalálod az API logot, a payment service logját és a DB latency metrikát.

11-20. témák

11. SQL indexing

Az index gyors keresési útvonal egy táblán. Jó mezők: amelyeken gyakran szűrsz, rendezel vagy joinolsz, például customerId, status, createdAt. Az index gyorsítja az olvasást, de lassítja az írást és helyet foglal.

Példa: Ha gyakran kérdezed le a customerId + status + createdAt alapján a renewals listát, ezekre kombinált index hasznos lehet.

12. Transaction

Több DB művelet egy logikai egységben. Vagy minden sikerül, vagy rollback. Payment, renewal vagy jogi státusz esetén fontos, hogy ne maradjon félig mentett állapot.

Példa: Renewal létrehozásakor egyszerre mented a renewal rekordot és a status history első bejegyzését. Ha az egyik bukik, rollback.

Interjún: For critical business operations I would use transactions where the data must change atomically.

13. Isolation level

Azt szabályozza, hogy párhuzamos tranzakciók mennyire látják egymás félkész módosításait. Magasabb isolation biztonságosabb, de több lockot és lassulást okozhat. Alacsonyabb gyorsabb, de dirty/phantom read jellegű gondok lehetnek.

Példa: Két admin egyszerre módosítja ugyanazt a rekordot. Magasabb isolation vagy optimistic locking segít elkerülni a furcsa versenyhelyeztet.

Interjún: It is a trade-off between consistency and performance.

14. SQL vs NoSQL

SQL jó relációkra, joinokra, tranzakciókra és erős konzisztenciára. NoSQL jó rugalmas schema, nagy scale, document/event/log jellegű adatok vagy key-based lookup esetén. A döntést nem divat, hanem access pattern alapján kell meghozni.

Példa: Customer, invoice, payment relációkra SQL jó. Nagy mennyiségű audit event vagy rugalmas document metadata tárolására NoSQL is szóba jöhet.

15. OAuth, OIDC, JWT

OAuth authorization: mit tehet az app vagy user. OIDC authentication réteg OAuth fölött: ki a user. JWT egy token formátum claim-ekkel, például userId, roles, scopes, expiry.

Példa: OIDC login után tudod, ki a user. OAuth scope alapján döntöd el, hívhatja-e a renewals API-t. JWT hordozhatja ezeket a claim-eket.

Interjún: OIDC tells who the user is, OAuth scopes define what they can access, JWT is often the token format.

16. API security

Nem csak login. Részei: auth, authorization, input validation, rate limiting, audit logging, secret management, least privilege és biztonságos error response. Backend validáció mindig kell, mert a frontend megkerülhető.

Példa: A backend ellenőrzi, hogy a user csak a saját customerId-jához tartozó portfolio adatokat láthatja, akkor is, ha más ID-t küld a requestben.

17. Conway's Law

A rendszer felépítése gyakran tükrözi a csapatok kommunikációját. Ha a csapatok rosszul határoltak, a service boundary-k is rosszak lehetnek. Service designnál ezért nem csak technikai, hanem ownership és kommunikációs szempont is kell.

Példa: Ha külön payment csapat és külön IP workflow csapat van, gyakran jobb ezek közé tiszta API boundary-t húzni, nem egy közös óriás service-t építeni.

18. TDD és testing strategy

TDD: előbb teszt, aztán kód. Nem mindig kell dogmatikusan, de segít tisztázni a várt működést. Testing stratégia: unit test kis logikára, integration test DB/API kapcsolatra, contract test API contractra, E2E teljes flow-ra.

Példa: Contract test jelzi, ha a backend átnevezi a status mezőt state-re, és ezzel eltöri a frontend vagy külső ügyfél integrációját.

19. System design document

Rövid döntéstámogató dokumentum. Tartalma: cél, scope, requirements, non-functional requirements, diagram, API contract, data model, kockázatok, alternatívák, rollout terv. Architect role-ban fontos, mert a designnak érthetőnek és vitathatónak kell lennie.

Példa: Egy design docban leírod, miért queue alapú async document generationt választasz sync API hívás helyett, és mik a rollback/monitoring lépések.

20. Scalability vs elasticity

Scalability: a rendszer tud nagyobb terhelést kezelni több erőforrással.

Elasticity: automatikusan fel-le skálázódik terhelés szerint. Kubernetes HPA például elasticitást adhat, ha jól vannak mérve a metrikák.

Példa: Horizontal scaling: 3 pod helyett 10 pod. Elasticity: a HPA automatikusan felviszi 10-re terheléskor, majd visszaveszi 3-ra.

Interjú: I care not only about scaling up, but also safe scaling down and cost control.

21-30. témák

21. Non-functional requirements

Nem azt írják le, mit csináljon a rendszer, hanem milyen minőségben: latency, availability, scalability, security, auditability, maintainability, cost. Architecture előtt ezeket kell tisztázni.

Példa: Nem elég annyi, hogy legyen renewals API. Tudni kell: napi mennyi request, mekkora latency elvárás, kell-e audit, mennyi downtime fér bele.

Interjú: Before choosing architecture I would clarify traffic, latency, security, audit and availability requirements.

22. Availability

Azt mutatja, mennyit elérhető a rendszer. 99.9% havi kb. 43 perc downtime körül van, 99.99% sokkal szigorúbb és drágább. Nem mindig kell maximum, business igény dönti el.

Példa: Ha a customer portal üzletileg kritikus, több instance, health check és DB failover kellhet. Egy belső admin riportnál lehet alacsonyabb cél.

23. Resilience

A rendszer hiba esetén sem omlik össze teljesen, hanem részlegesen tovább működik vagy gyorsan helyreáll. Példa: notification service hibája ne állítsa meg a renewal flow-t, csak később retryolja az emailt.

Példa: Ha az email service leáll, a renewal workflow ne bukjon el. Az email event queue-ban marad, később újrapróbálod.

24. Fault tolerance

Konkrét komponenshiba túlélése. Példa: kiesik egy pod, Kubernetes újraindítja; egy instance unhealthy, load balancer kiveszi. Resilience tágabb, fault tolerance konkrét hibák kezelése.

Példa: Kiesik egy pod, a load balancer nem küld rá több forgalmat, Kubernetes új podot indít, a rendszer tovább válaszol.

25. Caching

Gyakran olvasott adat gyors elérése. Lehet Redis, CDN vagy in-memory cache. Hasznos, de veszélyei vannak: stale data, invalidation, cache stampede. Előbb a read pattern és consistency igényt kell érteni.

Példa: Customer portfolio összesítő 5 percre cache-elhető Redisben, de payment státusz közvetlenül friss forrásból jön.

26. Cache invalidation

Annak eldöntése, mikor töröld vagy frissíted a cache-t. Megoldások: TTL, explicit invalidation update után, write-through, background refresh. Hibás invalidation régi üzleti adatot mutathat.

Példa: Ha a trademark státusz változik, töröld a kapcsolódó dashboard cache kulcsot, vagy rövid TTL-t használsz, hogy ne maradjon sokáig régi adat.

27. Message queue

Aszinkron feldolgozásra való. Producer betesz üzenetet, consumer később feldolgozza. Jó lassú munkákra, spike kezelésre, retryra és rendszerek leválasztására. Számolni kell duplicate üzenetekkel.

Példa: PDF generálás vagy külső partner sync nem blokkolja az API requestet. Az API üzenetet tesz queue-ba, worker dolgozza fel.

28. Event-driven architecture

A rendszerek eseményekre reagálnak, például RenewalRequested vagy PaymentCompleted. Előny: laza coupling és async flow. Hátrány: nehezebb debugging, eventual consistency és event versioning.

Példa: RenewalCreated event után a payment service, document service és notification service külön reagálhat, anélkül hogy egymást közvetlenül hívnák.

Interjú: Event-driven design is useful, but observability and idempotency must be explicit.

29. Eventual consistency

Az adat nem mindenhol friss azonnal, de rövid idő után beáll. Példa: payment után a dashboard pár másodperc múlva frissül. UI-ban ezt processing vagy pending állapottal kell kezelni.

Példa: Payment után a dashboard pár másodpercig Processing státuszt mutat, amíg a read model frissül.

30. Deployment strategies

Rolling update fokozatos csere. Blue-green két környezet gyors visszaállással. Canary kis forgalmon tesztel. Feature flag esetén a kód kint van, de a funkció kapcsolható. Cél: kisebb rollout kockázat.

Példa: Canary release: az új API verzió először csak a forgalom 5 százalékát kapja. Ha error rate nő, rollback.

31-40. témák

31. OOA / OOD

OOA azt jelenti, hogy elemzed a domain fogalmait és kapcsolatait. OOD azt jelenti, hogy ebből objektumokat, interface-eket, modulokat és felelősségeket tervezel. Tehát előbb megérted a business nyelvet, utána alakítod kóddá.

Példa: OOA: felismered a fogalmakat, például Customer, Portfolio, Trademark. OOD: ezekből modulokat/classokat és felelősségeket tervezel.

32. Design patterns

Design pattern egy bevált megoldási minta gyakori problémára. Nem cél önmagában használni őket. Akkor jók, ha egyszerűbbé teszik a kódot: például Strategy cserélhető algoritmushoz, Adapter külső rendszerek illesztéséhez, Repository adat-hozzáférés elrejtéséhez.

Példa: Adapter pattern: az egyik külső ügyfél XML-t küld, a másik JSON-t. Adapterek alakítják át őket közös belső modellre.

Interjú: I use patterns when they simplify the code, not just for the sake of patterns.

33. Layered architecture

Rétegek: controller/API, application/service, domain, infrastructure/data.

Előny: tiszta felelősségek, jobb tesztelhetőség. A controller ne tartalmazzon business logicot, csak request kezelést és service hívást.

Példa: Controller csak fogadja a POST /renewals requestet és meghívja a RenewalService-t. A business szabály nem a controllerben van.

34. Clean architecture

A domain/business logika ne függjön frameworktől, DB-től vagy UI-tól. Az infrastruktúra adaptereken keresztül kapcsolódik. Akkor hasznos, ha a domain elég komplex, és hosszú távú maintainability kell.

Példa: A domain logika nem közvetlenül Entity Framework vagy PostgreSQL részletekre épül, hanem repository/interface mögött van az adat-hozzáférés.

35. .NET / ASP.NET Core awareness

.NET awarenesshez nem kell expertnek tűnni. A lényeg: ASP.NET Core a modern web framework, C# a fő nyelv, EF Core az ORM, middleware a request pipeline része, dependency injection pedig alap működés.

Példa: ASP.NET Core-ban a controller fogadja a HTTP requestet, middleware kezeli például auth/loggingot, EF Core pedig DB műveleteket végez.

Interjú: My recent stack is Node.js/TypeScript, but I understand the .NET backend concepts and would ramp up on implementation details.

36. API Gateway vs Load Balancer

Load balancer forgalmat oszt instance-ok/podok között. API Gateway API kontrollpont: auth, rate limit, routing, transformation, versioning, monitoring. Külső API platformnál API Gateway gyakran hasznos.

Példa: Külső customer API előtt API Gateway kezeli az authot, rate limitet és loggingot. Belül a load balancer csak podok között osztja a forgalmat.

37. Health checks

Health endpoint jelzi, hogy egy instance kaphat-e forgalmat. Liveness: él-e még. Readiness: készen áll-e forgalomra. Zero downtime rolloutnál readiness különösen fontos.

Példa: Az új pod liveness szerint él, de readiness addig false, amíg nincs DB kapcsolat és config betöltve. Addig nem kap forgalmat.

38. Database migration

DB schema kontrollált változtatása. Biztonságos minta: új mező hozzáadása, app mindkét formát írja, backfill, app új mezőt olvas, régi mező később törölhető. Így nem töröd el a productiont.

Példa: Először hozzáadsz egy nullable newStatus mezőt, az app egy ideig mindkét mezőt kezel, backfill után vezeted ki a régit.

39. Distributed transaction / Saga

Több service-en átívelő üzleti folyamatnál nehéz egyetlen ACID transaction. Saga lépésekből áll, hibánál kompenzáló lépést indít. Példa: renewal, payment, document, notification flow.

Példa: Payment sikerült, de document generation bukott. Saga nem feltétlen mindent rollbackel, hanem FAILED_DOCUMENT státuszt és retry/kompenzációt indít.

40. Technical leadership

Nem csak people management. Követelményeket tisztázol, design opciókat hasonlítasz, riskeket kimondasz, feladatot kisebb deliverable-ökre bontasz, code reviewzol, unblockolod a csapatot és kommunikálsz product/infra oldallal.

Példa: Tech leadként egy nagy integration feature-t kisebb deliverable-ökre bontasz: API contract, mapping, validation, async worker, monitoring, rollout.

41-50. témák

41. API versioning

Az API változtatása úgy, hogy régi kliensek ne törjenek el. Lehet URL-ben, headerben vagy media type-ban. Új mező hozzáadása általában safe, meglévő mező törlése vagy átnevezése breaking change.

Példa: A /v1/renewals response megmarad, a /v2/renewals új mezőkkel vagy új struktúrával jön. Régi ügyfelek nem törnek el.

42. Backward compatibility

Az új verzió tovább működik a régi kliensekkel. Ha eddig status mezőt küldtél, ne cseréld hirtelen state-re. Inkább egy ideig mindkettőt támogasd, majd kommunikált deprecation után vezess ki.

Példa: Új optional field hozzáadása safe. A status mező átnevezése state-re breaking change, mert régi kliensek status-t várnak.

43. Deprecation

Régi API vagy mező még működik, de már nem ajánlott, és később megszűnik. Külső ügyfeleknél kell migration guide, határidő és kommunikáció. Hirtelen kikapcsolás rossz integrációs élmény.

Példa: A /old-renewal-status endpoint még működik 6 hónapig, de dokumentációban deprecated, és migration guide mutatja az új endpointot.

44. Rate limiting

Korlátozza, hogy user/customer/API key mennyi requestet küldhet időegység alatt. Védi a rendszert túlterhelés, abuse és költségrobbanás ellen. Tipikus válasz: HTTP 429 Too Many Requests.

Példa: Egy API key maximum 1000 request/minute hívást tehet. Ha túllépi, 429 Too Many Requests választ kap.

45. Throttling

A forgalom dinamikus visszafogása túlterhelés esetén. Rate limit gyakran fix szabály, throttling inkább adaptív lassítás. Cél: egy ügyfél vagy spike ne ölje meg az egész platformot.

Példa: Black Friday jellegű terhelésnél a rendszer dinamikusan lassítja a nem kritikus reporting endpointokat, hogy a core API stabil maradjon.

46. Circuit breaker

Ha egy downstream service sokszor hibázik, ideiglenesen nem hívod tovább. Megakadályozza, hogy retryokkal még jobban túlterheld a hibás dependency-t. Állapotok: closed, open, half-open.

Példa: Ha a payment service 50 százalékban hibázik, a circuit breaker open állapotba megy, és rövid ideig nem hívja tovább.

47. Cascading failure

Egy komponens hibája láncreakcióban ledönt másokat is. Példa: payment service lassú, API vár, requestek beragadnak, user retryol, még nagyobb terhelés jön. Védelem: timeout, circuit breaker, queue, backoff, rate limit.

Példa: Lassú downstream miatt API requestek beragadnak, elfogy a connection/thread pool, user retryol, még több request jön. Ez cascading failure.

48. Timeout

Maximum várakozási idő külső hívásra. Timeout nélkül requestek beragadhatnak, connection pool elfogyhat, cascading failure jöhet. Minden network callnál legyen explicit timeout.

Példa: Külső API hívásnál 3 másodperces timeout van. Ha lejár, pending státuszt vagy kontrollált hibát adsz, nem vársz örökké.

Interjú: Waiting forever is usually worse than failing predictably.

49. Bulkhead isolation

Erőforrások elkülönítése, hogy egy hibás rész ne vigye magával az egészet. Példa: külön queue document processingra, külön connection pool, külön worker pool payment hívásokra.

Példa: A document processing külön worker poolon fut. Ha túlterhelődik, nem viszi magával a public API request kezelést.

50. Fallback

Tartalék viselkedés hiba esetén. Példa: cache-ből régebbi adat, pending státusz, email későbbre rakása. Kritikus business műveletnél ne kamuzz sikert, inkább explicit pending vagy failed állapot legyen.

Példa: Ha a recommendation vagy reporting service nem elérhető, a core renewal flow megy tovább, csak a mellékes adat hiányzik vagy pending lesz.

51-60. témák

51. Authentication vs authorization

Authentication: ki vagy? Authorization: mit tehetsz? Login után tudjuk az identitást, de külön ellenőrizni kell, hogy az adott customer vagy portfolio adataihoz hozzáférhetsz-e.

Példa: Authentication: a token alapján Richard a user. Authorization: Richard hozzáfér-e a 123-as customer portfoliojához.

52. RBAC

Role-Based Access Control. Jogosultság szerepkör alapján: Admin, Support Agent, Customer, Read-only. Egyszerű és érthető, de sok kivételnél merevvé válhat.

Példa: Support Agent csak olvashat customer adatot, Finance User invoice/payment adatot kezelhet, Admin szélesebb joggal rendelkezik.

53. ABAC

Attribute-Based Access Control. Döntés tulajdonságok alapján: `user.companyId`, `document.sensitivity`, `customer.country`, `ownership`. Rugalmasabb, de bonyolultabb tesztelni és debugolni.

Példa: User csak akkor láthat dokumentumot, ha `user.companyId == document.companyId` és `document.sensitivity` nem `restricted`.

54. Least privilege

Minden user, service és process csak a minimális jogosultságot kapja, ami kell. Egy reporting service például ne tudjon törölni, ha csak olvasnia kell. Service-to-service authnál scoped tokenek hasznosak.

Példa: Egy notification service csak email eventet olvashat és küldhet, de nem módosíthat payment vagy trademark adatot.

55. Secrets management

Secret: DB password, API key, JWT signing secret, OAuth client secret, private key. Ne legyen Gitben vagy logban. Használj secret managert, env változót, Kubernetes Secretet vagy Vault/AWS Secrets Manager jellegű megoldást.

Példa: DB password és OAuth client secret nem Gitben van, hanem Kubernetes Secretben vagy AWS Secrets Managerben, és logba sem kerül.

56. Encryption in transit

Az adat hálózaton titkosítva megy, tipikusan TLS/HTTPS használatával. Ez védi a tokeneket, személyes adatokat és üzleti adatokat network sniffing ellen.

Példa: Browser -> API HTTPS-en megy, service-to-service kommunikáció is TLS-sel védett, így tokeneket nem lehet simán lehallgatni.

57. Encryption at rest

Az adat tárolás közben titkosított: DB, disk, backup, object storage. IP, payment és ügyféladatoknál fontos. Nem helyettesíti az authorizationt, csak egy védelmi réteg.

Példa: A database disk, backup és object storage titkosított. Ha valaki storage szinten fér hozzá, nem olvassa plain textben az adatot.

58. Audit logging

Olyan napló, amely bizonyítja, ki mit csinált. Példa: ki módosított trademark adatot, ki nézett meg customer portfóliót, melyik API key hívott endpointot. Különbözik a debug logtól.

Példa: Audit logban látszik: ki módosította a trademark státuszt, mikor, milyen régi és új értékkel, milyen requestId mellett.

59. Input validation

Minden bejövő adat ellenőrzése backend oldalon: required mezők, típus, hossz, enum, dátum, jogosultság. Frontend validáció nem elég, mert az API közvetlenül hívható.

Példa: A backend ellenőrzi, hogy renewalDate valid dátum, customerId létezik, status enum ismert, string hosszak nem túl nagyok.

60. Schema evolution

Adatstruktúra változtatása kompatibilisen. Új mező hozzáadása általában safe. Mező törlése, átnevezése, típusváltása vagy enum váratlan módosítása veszélyes. Data exchange platformnál kritikus.

Példa: Régi eventben csak status van, új eventben statusReason is. Consumernek el kell viselnie, ha az új mező hiányzik.

61-70. témák

61. OpenAPI / Swagger

OpenAPI géppel és emberrel is olvasható REST API leírás. Swagger UI gyakran ennek vizuális dokumentációja. Külső API platformnál ez azért fontos, mert az ügyfél nem belső kódot lát, hanem a contractból integrál.

Példa: Swagger UI-ban a partner látja a POST /renewals request body-t, response példát és hibakódokat, így könnyebb integrálnia.

62. DTO

Data Transfer Object. Az API-n küldött/fogadott objektum. Nem feltétlen ugyanaz, mint a DB vagy domain model. Előnye: nem szivárogtatod ki a belső modellt, és stabilabb API contractot tarthatsz.

Példa: RenewalRequestDto-ban csak customerId és trademarkId van, de a DB entity tartalmaz internal notes és audit mezőket is. Ezeket nem küldöd ki.

63. Domain model vs database model

Domain model business fogalmakat és szabályokat ír le. Database model a tárolási formát. Egy Renewal domain objektum mögött több tábla is lehet: renewals, payments, documents, status_history.

Példa: Domainben Renewal státuszváltási szabályai vannak. DB-ben külön renewals, payments és renewal_status_history táblák lehetnek.

64. Validation vs business rule

Validation formai/technikai ellenőrzés: email, required mező, enum.

Business rule üzleti szabály: jogosult-e az ügyfél, megújítható-e a trademark, rendezett-e a payment. Külön hibakezelést igényelnek.

Példa: Validation: a date mező formátuma rossz. Business rule: a trademark nem újítható, mert már expired vagy nincs jogosultság.

65. Canonical data model

Közös belső adatmodell, amire a különböző ügyfélformátumokat átalakítod. Így a belső rendszer nem ismeri minden ügyfél egyedi mezőneveit és formátumait.

Példa: Két partner más mezőneveket küld, de belül mindent TrademarkRenewal modellre alakítasz: trademarkId, dueDate, customerId.

66. Data mapping

Egy adatstruktúra átalakítása másikra. Tipikus gondok: dátumformátum, enum, country code, currency, optional/required mezők. Mapping logika legyen explicit, tesztelt és versioned.

Példa: Egy partner DE országcódot küld, másik Germany stringet. Mapping normalizálja ISO country code-ra.

67. Webhook

A rendszer esemény esetén meghívja az ügyfél URL-jét. Példa: renewal státusz változáskor POST customer webhookra. Kell hozzá retry, signature validation, idempotency, timeout és delivery log.

Példa: RenewalCompleted eventnél a rendszer POST-ol az ügyfél webhook URL-jére. Ha timeout, retryol és naplózza a delivery státuszt.

68. Polling vs webhook

Polling: kliens időnként lekérdez. Webhook: rendszer szól változáskor. Polling egyszerűbb, de pazarló lehet. Webhook hatékonyabb, de bonyolultabb retry/security/delivery tracking miatt.

Példa: Polling: partner percenként hívja a status endpointot. Webhook: te szólsz neki, amikor státusz változott.

69. Cursor-based pagination

Pagination nagy lista oldalakra bontása. Offset egyszerű, de változó listánál duplikáció vagy kihagyás lehet. Cursor stabilabb nagy és folyamatosan változó adatoknál, ezért external API-khoz gyakran jobb.

Példa: GET /renewals?cursor=abc&limit=50 stabilabban működik nagy, folyamatosan változó listán, mint offset=5000.

70. Optimistic locking

Mentéskor ellenőrzöd, hogy az adat közben változott-e. Példa: version mező. Ha kliens version 7 alapján mentene, de DB már version 8, akkor 409 Conflict. Így nem írod felül csendben más módosítását.

Példa: Két admin ugyanazt a rekordot szerkeszti. A második mentés 409 Conflict, mert a version már megváltozott.

71-80. témák

71. REST vs RPC

REST resource-központú API-stílus, RPC művelet-központú. Egyik sem mindig jobb. Külső, dokumentált integrációhoz REST gyakori, belső nagy teljesítményű service kommunikációhoz gRPC/RPC lehet praktikusabb.

Példa: REST: POST /renewals. RPC: POST /submitRenewal. Külső API-nál REST könnyebben dokumentálható OpenAPI-val.

72. gRPC és Protocol Buffers

gRPC gyors service-to-service kommunikáció. Protobuf típusos, bináris üzenetformátum. Előny: gyors, strict contract, kódgenerálás. Hátrány: külső ügyfélnek kevésbé kényelmes, debugging nehezebb, mint JSON-nál.

Példa: Belső payment-service hívás gRPC-vel gyors és típusos lehet, de külső partnernek REST/JSON kényelmesebb.

73. Synchronous vs asynchronous communication

Sync: hívó vár a válaszra. Egyszerű, de lassú dependency blokkolhat. Async: queue/event indítja a folyamatot, az API pending státuszt adhat. Lassú vagy unreliable downstreamnál gyakran jobb async.

Példa: Az API azonnal validálja a requestet, majd queue-ba teszi a hosszú document generationt, és PENDING választ ad.

74. Service discovery

Microservice-eknek meg kell találniuk egymást változó pod/instance IP-k mellett. Kubernetesben service név alapján hívsz, például payment-service, az infra route-ol az aktuális podokra.

Példa: A renewal-api nem konkrét IP-t hív, hanem `http://payment-service` címet. Kubernetes mögötte aktuális podra route-ol.

75. Reverse proxy

Köztes réteg kliens és app között: NGINX, API Gateway, Ingress. Feladata lehet routing, TLS termination, compression, auth előszűrés, rate limit, load balancing és request limit.

Példa: NGINX vagy Ingress terminálja a TLS-t, limitálja a request méretet, majd továbbítja az API service felé.

76. Kubernetes Ingress

Megmondja, hogy külső HTTP/HTTPS forgalom hogyan jusson be a clusterbe. Példa: `api.company.com/renewals` -> `renewal-api-service`. Service inkább belső pod elérés, Ingress külső routing.

Példa: `api.dennemeyer.example/renewals` a `renewal service-re`, `/payments` a `payment service-re` route-ol Ingress szabállyal.

77. Load balancing

Forgalom szétosztása több instance/pod között. Növeli kapacitást és availabilityt, segíti rolling update-et és unhealthy instance kivételét. Stateless service-ekkel sokkal egyszerűbb.

Példa: Három API pod fut. Ha az egyik readiness false, a load balancer csak a másik kettőre küld forgalmat.

78. Stateless vs stateful service

Stateless service nem tart kritikus állapotot memóriában, bármelyik pod kezelheti a requestet. Stateful service állapotot tart, például WebSocket connection vagy in-memory session. Stateful skálázás nehezebb.

Példa: JWT-s REST API stateless. WebSocket chat stateful, mert a kapcsolat konkrét instance-hoz kötődik, ezért Redis pub/sub segíthet.

79. Session management

Server-side sessionnél szerver tárolja a sessiont, kliens session ID-t kap. JWT-nél token hordoz claim-eket, backend aláírást ellenőriz. JWT stateless API-hoz jó, de revoke, expiry és key rotation fontos.

Példa: JWT access token 15 percig él, refresh tokennel újítható. Kilépés/revoke kezeléshez külön stratégia kell.

80. Horizontal vs vertical scaling

Vertical: erősebb gép. Horizontal: több instance/pod. Horizontal cloud/K8s környezetben rugalmasabb, de az appnak statelessnek vagy jól külső állapottal tervezettnak kell lennie.

Példa: Először nézed a bottlenecket. Ha CPU-bound az API, több pod segíthet. Ha DB a szűk keresztmetszet, több pod csak ront.

81-90. témák

81. N+1 query problem

Egy lista lekérése után minden elemhez külön query fut. Példa: 1 query 100 renewalra, majd 100 query customer adatokra. Megoldás: join, batch loading, eager loading, GraphQL DataLoader jellegű batching.

Példa: 100 renewal sorhoz nem 100 külön customer query kell, hanem join vagy batch lekérdezés customerId-k alapján.

82. Connection pool

DB kapcsolatok készlete. Nem nyitsz minden requesthez új kapcsolatot, mert drága. Túl kicsi pool várakozást okoz, túl nagy pool túlterheli a DB-t. Több podnál a pool méretek összeadódnak.

Példa: 10 pod x 50 DB connection = 500 kapcsolat. Ez lehet túl sok a DB-nek, ezért pool méretet globálisan kell gondolni.

83. Strong consistency

Sikeres írás után minden olvasás azonnal friss adatot lát. Fontos payment, jogosultság, jogi státusz vagy kritikus üzleti döntés esetén. Drágább vagy nehezebben skálázható lehet distributed rendszerben.

Példa: Payment sikeres mentése után a következő payment status lekérdezésnek azonnal PAID-et kell mutatnia, nem régi értéket.

84. Eventual consistency

Az adat nem mindenhol friss azonnal, de idővel konzisztens lesz. Dashboard, read model vagy analytics esetén elfogadható lehet. UI-ban látszódjon, ha folyamatban van a frissülés.

Példa: A dashboard read model 2 másodperc késéssel frissül, de a user Processing státuszt lát, nem ellentmondásos sikert.

85. CAP theorem

Distributed rendszerben hálózati partition mellett választani kell consistency és availability között. Paymentnél inkább consistency, dashboard/analytics esetén gyakran availability + eventual consistency elfogadható.

Példa: Hálózati szakadásnál a rendszer vagy tovább válaszol esetleg régebbi adattal, vagy megállít bizonyos műveleteket a konzisztencia védelmében.

86. Sharding / partitioning

Adat szétoztása több node vagy DB között. Kell, ha egy DB már nem bírja az adatot vagy forgalmat. Nehézségek: shard key, hot shard, cross-shard query, rebalancing. Csak indokolt esetben érdemes.

Példa: CustomerId hash alapján több shardra osztod a renewals adatot, ha egyetlen DB már nem bírja a forgalmat.

87. Hot partition / hot shard

Egy shard aránytalanul több forgalmat kap. Rossz shard key példáulul country, ha a forgalom nagy része egy országból jön. Jó kulcs jó eloszlást ad, de még támogatja a fontos lekérdezéseket.

Példa: Ha tenantId alapján shardolsz, egy óriási ügyfél hot shardot okozhat. Hash vagy al-bontás segíthet.

88. Replication és read replica

Primary DB ír, read replica olvas. Jó olvasási terhelés skálázására és reporting leválasztására. Figyelni kell replica lagre: friss írás nem biztos, hogy azonnal látszik a replicán.

Példa: Dashboard olvas read replicáról, de közvetlen payment után a friss státuszt primaryról olvasod a replica lag miatt.

89. At-least-once delivery

Queue/event rendszerekben gyakori garancia: az üzenet legalább egyszer megérkezik, tehát duplikálódhat. Consumer legyen idempotens, hogy második feldolgozás ne okozzon duplikált üzleti hatást.

Példa: Ugyanaz a PaymentCompleted event kétszer jön. A consumer látja, hogy eventId már processed, és nem küld két invoice emailt.

90. Outbox pattern

DB változás és event küldés megbízható összekötése distributed transaction nélkül. Ugyanabban DB tranzakcióban mented az üzleti adatot és az outbox eventet, majd worker publikálja az eventet.

Példa: Renewal mentésekor ugyanabban transactionben outbox táblába is bekerül a RenewalCreated event. Worker később publikálja.

91-100. témák

91. Latency vs throughput

Latency: egy request válaszüzeje. Throughput: adott idő alatt feldolgozott request/üzenet mennyisége. Lehet magas throughput, de rossz user élmény, ha requestek sokat várnak queue-ban.

Példa: Egy API lehet képes 5000 req/sec throughputra, de ha p95 latency 2s, a user élmény rossz.

92. P95 / P99 latency

P95: a requestek 95 százaléka ennél gyorsabb, 5 százalék lassabb. Átlag latency félrevezető lehet, mert a tail latency mutatja, hogy a leglassabb userek mennyire szenvednek.

Példa: Átlag 80 ms, de p99 3s. Ez azt jelzi, hogy kevés user nagyon lassú választ kap, amit átlagból nem látnál.

93. Bottleneck

A rendszer szűk keresztmetszete. Lehet DB query, index hiány, connection pool, CPU, memória, network, external API, lock contention vagy túl sok log. Előbb mérni kell, csak utána skálázni.

Példa: Lassú endpointnál először megnézed DB query időt, external API latencyt, CPU-t, memoryt és lockokat, nem rögtön több szervert veszel.

94. Backpressure

A rendszer jelzi, hogy nem bír több munkát, és lassít vagy visszautasít. Megoldás: queue limit, 429, rate limit, async processing, autoscaling. Jobb kontrolláltan lassítani, mint összeomlani.

Példa: Ha a queue age nő, az API 202 Accepted + pending választ ad, vagy korlátozza az új feladatokat, hogy ne omoljon össze.

95. Graceful shutdown

Leálláskor az app nem hal meg azonnal, hanem nem fogad új requestet, befejezi az aktívakat, lezárja DB/Redis kapcsolatokat, majd kilép. Rolling deploymentnél nagyon fontos.

Példa: Deploy közben a pod readiness false-ra vált, befejezi az aktív requesteket, lezárja kapcsolatokat, csak utána áll le.

96. Correlation ID

Egy egyedi request azonosító, amit végigviszel több service-en. Hiba esetén ezzel keresed vissza az összes kapcsolódó logot. Microservice és API platform debuggingban alap.

Példa: requestId=abc-123 minden logban megjelenik az API, payment és document service-ben, így egy ügyfélhibát végig tudsz követni.

97. Structured logging

Nem sima szöveget logolsz, hanem mezőket: event, customerId, renewalId, requestId, reason. Így Kibana/Grafana/Loki/Datadog jellegű toolokban könnyen kereshető és szűrhető.

Példa: JSON logban mezők vannak: event=payment_failed, customerId=123, requestId=abc, reason=timeout. Erre könnyű keresni.

98. Alerting

Monitoring nézés, alerting értesítés. Jó alert actionözhető: error rate, p95 latency, queue age, failed jobs. Rossz alert túl zajos, például önmagában CPU 80%, ha nem vezet konkrét teendőhöz.

Példa: Actionözhető alert: error rate 5 százalék felett 10 percig. Nem actionözhető: egyszeri CPU spike.

99. SLO / SLA / SLI

SLI a mért mutató: latency, error rate, availability. SLO belső cél: például 99.9% successful API response. SLA ügyfél felé vállalt szerződéses ígéret, akár kompenzációval.

Példa: SLI: p95 latency. SLO: p95 legyen 500 ms alatt a hónap 99 százalékában. SLA: ügyfélnek szerződésben vállalt szint.

100. Error budget

Az SLO alapján megengedett hiba mennyisége. Ha van error budget, mehet gyorsabb release. Ha elfogyott, reliability fixek és incident megelőzés a prioritás. Összeköti a technikai döntést business kockázattal.

Példa: Ha az error budget elfogyott, új feature rollout helyett reliability fix, alert tuning vagy incident utómunka kerül előre.

Záró pozicionálás

A legfontosabb narratíva: nem .NET-first architectként kell eladnod magad, hanem senior full-stack / backend-focused engineer-ként, aki production rendszerekben már csinált architecture-level munkát: API contractok, service boundary-k, scalability trade-offok, rollout, observability, security és mentoring.

Mondható verzió angolul:

My recent production stack is mainly Node.js and TypeScript, but many of my responsibilities were architecture-level: API contracts, service design, reliability, observability, deployment safety and mentoring. I would need ramp-up time on .NET implementation details, but the architecture and platform problems are familiar.

Forrásalap: feltöltött Denneweyer Software Architect álláshirdetés és Richard Sallai Full Stack CV.